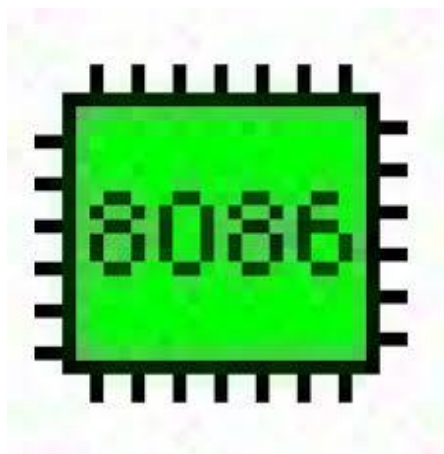




گزارش کار پروژه

پروژه: تبدیل عدد دسیمال به باینری و تبدیل عدد باینری به دسیمال

درس: زبان ماشین و اسمبلی

استاد: دکتر علیرضا قدیری

تهیه شده توسط حسن صمدی و علی فاضل نیا

آذر ماه ۱۴۰۳

مقدمه

باسمه تعالی

تبدیل اعداد از یک سیستم عددی به سیستم عددی دیگر، یکی از مباحث اساسی در علوم کامپیوتر و برنامه‌نویسی به شمار می‌رود. سیستم اعداد دسیمال (Decimal) که مبتنی بر مبنای ۱۰ است و سیستم اعداد باینری (Binary) که از مبنای ۲ استفاده می‌کند، دو مورد از پرکاربردترین سیستم‌های عددی هستند. سیستم دسیمال به دلیل استفاده گسترده در زندگی روزمره و امور حسابداری، و سیستم باینری به عنوان زبان اصلی ماشین‌ها و پردازنده‌ها، اهمیت ویژه‌ای دارند. در نتیجه، تبدیل اعداد میان این دو سیستم، نه تنها یک مهارت بنیادی برای برنامه‌نویسان است، بلکه در توسعه الگوریتم‌های کارآمد برای پردازش داده‌ها و طراحی سیستم‌های رایانه‌ای نیز نقشی کلیدی ایفا می‌کند.

در این گزارش، تمرکز ما بر پیاده‌سازی الگوریتم تبدیل عدد دسیمال به باینری و برعکس با استفاده از زبان اسمبلی در محیط emu8086 و پردازنده x8086 است. این پردازنده به عنوان یکی از نخستین معماری‌های پرکاربرد در دنیای کامپیوترهای شخصی، همچنان به عنوان یک ابزار آموزشی قدرتمند در درک مفاهیم سطح پایین و معماری سیستم مورد استفاده قرار می‌گیرد. گزارش حاضر به چهار بخش اصلی تقسیم شده است:

۱. **آشنایی با سیستم‌های عددی دسیمال و باینری و الگوریتم تبدیل آنها:** در این بخش، مفاهیم اولیه مرتبط با اعداد در مبنای‌های ۱۰ و ۲ توضیح داده شده و فرآیند الگوریتمی تبدیل این دو سیستم به یکدیگر با جزئیات تشریح می‌شود. این بخش شامل اصول تئوری نیز خواهد بود.
۲. **معرفی دستورات زبان اسمبلی و محیط x8086:** با توجه به اینکه پیاده‌سازی الگوریتم‌ها در سطح اسمبلی نیازمند دانش دقیق از دستورات و معماری پردازنده است، در این بخش دستورات و ساختارها و مفاهیم مورد استفاده در برنامه توضیح داده شده و ارتباط آن‌ها با معماری پردازنده x8086 بررسی می‌شود.
۳. **تحلیل کدهای نوشته‌شده و توضیح خط به خط آنها:** این بخش شامل تشریح دقیق کدهای نوشته‌شده برای پیاده‌سازی تبدیل اعداد دسیمال به باینری و بالعکس است. هر خط کد به همراه هدف و عملکرد آن توضیح داده شده است.
۴. **منابع و مراجع:** در پایان، منابع مورد استفاده برای گردآوری اطلاعات و پیاده‌سازی الگوریتم معرفی خواهند شد. این منابع شامل مراجع معتبر علمی و آموزشی برای معماری x8086 و زبان اسمبلی خواهند بود.

هدف از نگارش این گزارش، ارائه یک راهنمای جامع برای درک و پیاده‌سازی الگوریتم‌های تبدیل اعداد در سطح زبان ماشین است. این موضوع می‌تواند برای دانشجویان رشته کامپیوتر، برنامه‌نویسان علاقه‌مند به معماری سیستم، و همچنین افراد فعال در حوزه طراحی و بهینه‌سازی نرم‌افزارهای سطح پایین مفید واقع شود. علاوه بر این، تحلیل دقیق الگوریتم و کدها می‌تواند به درک بهتر مفاهیم پایه‌ای پردازنده‌ها و معماری آن‌ها کمک کند.

«بخش اول»

در این بخش به معرفی اعداد دسیمال و باینری و فرآیند الگوریتمی تبدیل این دو به هم خواهیم پرداخت

اعداد باینری:

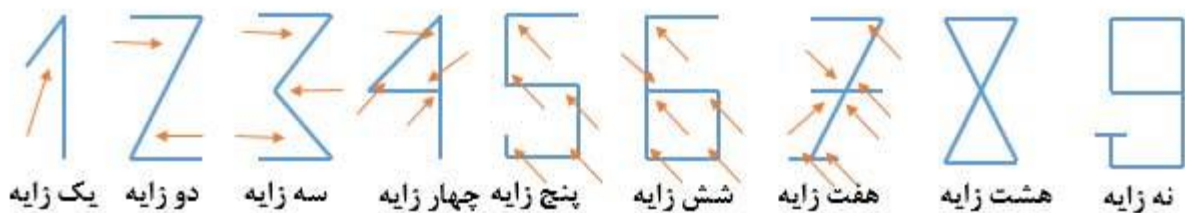
پرباربردترین سیستم عددی در دیجیتال، سیستم شماره گذاری باینری است. به طور کلی، دستگاه اعداد باینری و دستگاه اعداد دسیمال از قوانین مشابهی تبعیت می کنند، با این تفاوت که برخلاف دستگاه دسیمال که از توان های ۱۰ استفاده می کند، در دستگاه باینری از توان های ۲ استفاده می شود. سیستم های دیجیتالی و کامپیوتری برای نمایش یک وضعیت، تنها از دو مقدار «۰» منطقی و «۱» منطقی استفاده می کنند. این «۰» و «۱» ها در مبنای ۲ بوده و یا ارقام باینری هستند.

یک عدد باینری مثل ۱۰۱۱۰۰۱۰۱ به کمک رشته ای از «۱» ها و «۰» ها نشان داده می شود که با حرکت از سمت راست به چپ، هر رقم ارزشی دو برابر رقم قبلی دارد. اگر بخواهیم عدد باینری را در فرم $N = b_i q^i$ نشان دهیم، از آن جایی که هر رقم می تواند تنها دو مقدار مختلف اختیار کند، q برابر ۲ خواهد بود.

در اعداد باینری نیز هر رقم وزن مخصوص به خود را دارد. در این اعداد سمت چپ ترین بیت، پربارزش ترین بیت است که به آن (Most Significant Bit: MSB) گفته می شود. همچنین سمت راست ترین بیت عدد، کم ارزش ترین بیت آن است که (Least Significant Bit: LSB) نامیده می شود.

اعداد دسیمال یا دهدهی:

اعداد در این مبنا دارای ۱۰ رقم معنی دار ۰، ۱، ...، ۹ می باشند. دلیل استفاده از علامت های ۰، ۱، ...، ۹ برای این نه رقم معنی دار به تعداد شکستگی ها (زوایا) در نمایش آنها برمی گردد.



در هر عدد دسیمال، با حرکت از سمت راست به چپ ارقام به ترتیب مقدار یکان، دهگان، صدگان، هزارگان و ... دارند که معادل ریاضی این مقادیر به صورت $۱۰^۰$ ، $۱۰^۱$ ، $۱۰^۲$ ، $۱۰^۳$ و ... است. در اعداد اعشاری وزن هر مکان در سمت چپ ممیز توان مثبتی از ۱۰ است؛ به همین ترتیب، در سمت راست ممیز و با حرکت از چپ به راست، وزن هر رقم توان‌های منفی ۱۰ مثل 10^{-1} ، 10^{-2} ، 10^{-3} و ... است.

به راحتی می‌توان فهمید «سیستم شماره‌گذاری دسیمال» از مبنای ۱۰ استفاده می‌کند و مکان هر رقم نشان‌دهنده اهمیت یا وزن آن رقم است. اگر بخواهیم یک عدد دسیمال را با فرمول گفته شده در بالا نشان دهیم، q برابر ۱۰ خواهد بود. به عنوان مثال، ۲۰ معادل $2 * 10^1$ و یا ۴۰۰ معادل $4 * 10^2$ است.

با ضرب هر یک از ارقام عددی دسیمال در وزن مخصوصش و جمع تمامی اعداد تولید شده، معادل همان عدد به دست می‌آید. مثلاً $N = ۶۱۶۳$ مساوی است با:

$$۶۰۰۰ + ۱۰۰ + ۶۰ + ۳ = ۶۱۶۳$$

برای تأکید بیشتر بر وزن ارقام می‌توان به شکل زیر نیز نشان داد:

$$(۶ \times ۱۰۰۰) + (۱ \times ۱۰۰) + (۶ \times ۱۰) + (۳ \times ۱) = ۶۱۶۳$$

همچنین در صورت استفاده از توان‌های ۱۰ داریم:

$$(۶ \times ۱۰^۳) + (۱ \times ۱۰^۲) + (۶ \times ۱۰^۱) + (۳ \times ۱۰^۰) = ۶۱۶۳$$

همانند اعداد باینری، در دستگاه اعداد دهدهی یا دسیمال، سمت چپ‌ترین رقم، پر ارزش‌ترین رقم است که به آن (Most Significant Digit: MSD) نیز گفته می‌شود. همچنین سمت راست‌ترین رقم یک عدد، کم ارزش‌ترین رقم آن است که (Least Significant Digit: LSD)

نیز خوانده می‌شود. برای مثال قبل، عدد ۶ پر وزن‌ترین رقم یا همان MSD و عدد ۳ کم وزن‌ترین رقم یا LSD است.

حال که به معرفی اعداد باینری و دسیمال پرداختیم به سراغ الگوریتم تبدیل این دو نوع اعداد به هم می‌پردازیم.

تبدیل اعداد باینری به دسیمال:

ابتدا ارزش مکانی رقم‌ها را محاسبه نموده. بدین طریق که رقم اول از سمت راست در جایگاه $(2)^0$ و رقم دوم در جایگاه $(2)^1$ و ... می‌باشند. سپس هر کدام از رقم‌ها را در ارزش مکانی آن ضرب کرده و همه را با هم جمع می‌کنیم (جمع در مبنای ده) عدد به دست آمده در مبنای ده و برابر عدد ابتدایی در مبنای دو است.

مثال:

$$100101_2 = [(1) \times 2^5] + [(0) \times 2^4] + [(0) \times 2^3] + [(1) \times 2^2] + [(0) \times 2^1] + [(1) \times 2^0]$$

$$100101_2 = [1 \times 32] + [0 \times 16] + [0 \times 8] + [1 \times 4] + [0 \times 2] + [1 \times 1]$$

$$100101_2 = 37_{10}$$

تبدیل اعداد دسیمال به باینری:

یکی از روش‌های تبدیل از مبنای ده‌دهی به دودویی تقسیم متوالی بر عدد دو است؛ که طی آن باقی‌مانده‌ها مورد استفاده قرار می‌گیرند. تقسیمات را تا صفر شدن خارج قسمت انجام می‌دهیم. در نهایت آخرین خارج قسمت و بعد از آن باقی‌مانده‌ها را از آخر به اول کنار هم‌دیگر قرار می‌دهیم. عدد به دست آمده معادل دودویی خواهد بود.

مثال برای عدد ۳۲:

۳۲		
۳۲/۲	۱۶	•
۱۶/۲	۸	•
۸/۲	۴	•
۴/۲	۲	•
۲/۲	۱	•
۱		

حالا اعداد را از پایین به بالا می‌خوانیم: ۱۰۰۰۰۰

«بخش دوم»

در این بخش به معرفی دستورات و مفاهیم استفاده شده در پیاده سازی این الگوریتم در زبان اسمبلی و سیستم پردازنده x8086 در محیط emu8086 خواهیم پرداخت.

کاربرد دستورات و قالب‌ها	دستورات و مفاهیم
ذخیره و بازیابی مقادیر رجیسترها	PUSH & POP
جابه‌جایی مقادیر بین رجیسترها و حافظه	MOV
فراخوانی وقفه‌ها (interrupts) ، برای مثال 10H و 21H برای ورودی / خروجی	INT
مقایسه مقادیر	CMP
انجام پرش‌ها و تصمیم‌گیری‌ها	JE & JNE & JMP
فراخوانی و بازگشت به Procedures	CALL & RET
برای بارگذاری آدرس حافظه به یک ریجستری	LEA
برای ضرب و تقسیم	MUL & DIV
برای تفریق و جمع	SUM & ADD
برای شیفت به چپ	SHL
دستور یا مجموعه‌ای از دستورات است که می‌توان آن را به صورت یک واحد مشخص فراخوانی کرد.	Macro
مشابه مفهوم تابع در سایر زبان‌های برنامه‌نویسی	Procedure
قالبی که مشخص می‌کند که برنامه از مدل حافظه کوچک استفاده می‌کند، که در آن کد و داده‌ها هر دو در یک بخش 64 KB قرار دارند.	.model small
شروع بخش داده‌ها را اعلام می‌کند، جایی که متغیرها و ثابت‌ها تعریف می‌شوند.	.data
شروع بخش کدها را اعلام می‌کند. جایی که دستورات تعریف می‌شوند.	.code
ناحیه‌ای از حافظه که برای ذخیره موقت داده‌ها در هنگام انتقال یا پردازش استفاده می‌شود.	Buffer (بافر)

Label (لیبل)	نام نمادین برای اشاره به یک آدرس حافظه. همچنین مرجعی برای پرش‌ها و فراخوانی‌ها یا داده‌ها.
Null terminator	یک کاراکتر خاص (0 یا 0\0 باینری) که برای علامت‌گذاری استفاده می‌شود. این کاراکتر به برنامه کمک می‌کند انتهای رشته‌های متنی را تشخیص دهد.

«بخش سوم»

در این بخش خط به خط کدهای نوشته شده را بررسی خواهیم کرد.

```

ORG 100h

```

در اینجا تعیین میشود که از آدرس 100 برنامه اجرا شود در واقع یک پیش فرض برای برنامه های COM است .

```

PUTC      MACRO char
            PUSH AX
            MOV AL, char
            MOV AH, 0Eh
            INT 10h
            POP AX
ENDM

```

تعریف ماکرو (Macro): یک دستور یا مجموعه‌ای از دستورات است که می‌توان آن را به صورت یک واحد مشخص فراخوانی کرد. ماکروها این امکان را می‌دهند که بخش‌هایی از کد را برای استفاده‌های مکرر تعریف می‌شود، به راحتی آن‌ها را در قسمت‌های مختلف برنامه فراخوانی و استفاده شود.

PUTC: نام ماکرو است.

Char: پارامتری برای ماکروی مورد نظر که نمایانگر کاراکتری است که قرار است چاپ شود.

PUSH AX: مقدار رجیستر AX را در پشته ذخیره می‌کند.

MOV AL, char: کاراکتر مورد نظر را به رجیستر AL منتقل می‌کند.

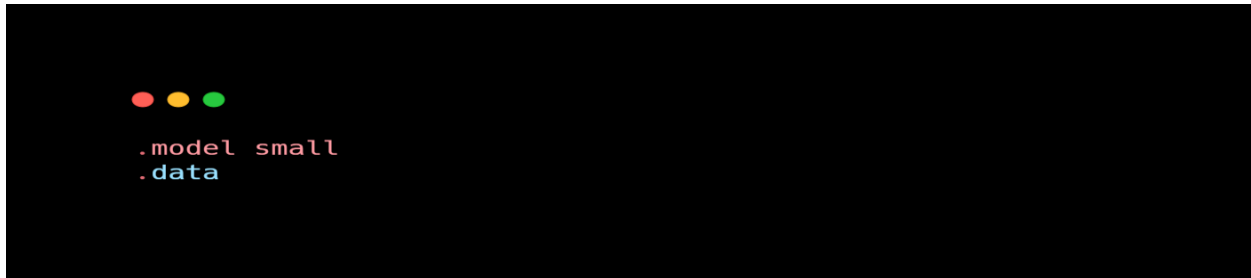
MOV AH, 0Eh: تابع نمایش کاراکتر در داس را تنظیم می‌کند.

INT 10h: وقفه داس 10h را برای نمایش کاراکتر فراخوانی می‌کند.

POP AX: مقدار رجیستر AX را از پشته بازگردانی می‌کند.

ENDM: این کلمه کلیدی پایان تعریف ماکرو را نشان می‌دهد. از این نقطه به بعد، می‌توانیم از ماکرو **PUTC** در هر جایی از کد خود استفاده کنیم.

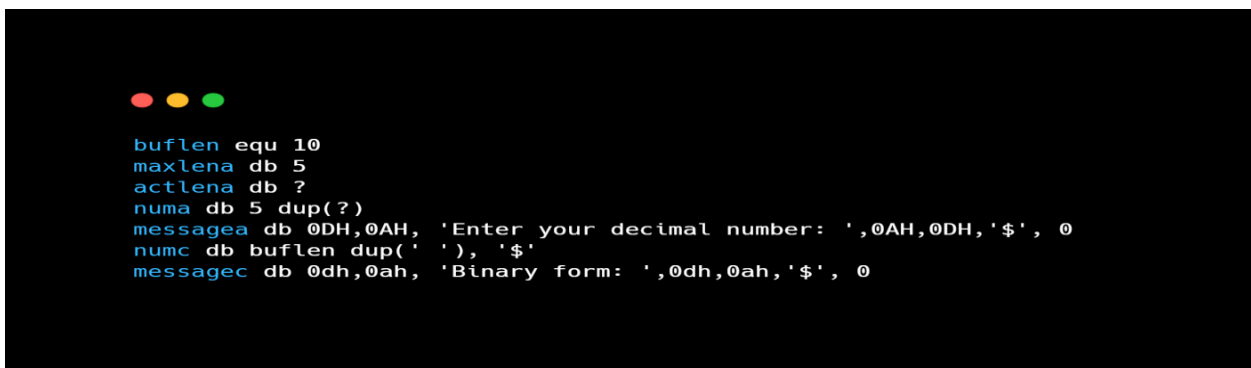
تعریف مدل حافظه و بخش داده‌ها:



model small: مشخص می‌کند که برنامه از مدل حافظه کوچک استفاده می‌کند، که در آن کد و داده‌ها هر دو در یک بخش 64 KB قرار دارند.

data: شروع بخش داده‌ها را اعلام می‌کند. از اینجا به بعد به تعریف ثابت‌ها و متغیرها می‌پردازیم.

تعریف متغیرها و متن پیام‌ها :



buflen equ 14: ثابتی با نام **buflen** را به 10 تنظیم می‌کند، که طول بافر برای ذخیره اعداد ورودی را مشخص می‌کند.

maxlena db 5: یک بایت با نام **maxlena** با مقدار اولیه 5 تعریف می‌کند که نشان‌دهنده حداکثر طول یک عدد دسیمال است.

actlena db ?: یک بایت با نام **actlena** با مقدار اولیه نامشخص تعریف می‌کند که بعداً مقدار دهی می‌شود.

numa db 5 dup(?): یک بافر 5 بایتی numa با مقادیر اولیه نامشخص تعریف می‌کند که برای ذخیره عدد دهدهی کاربر استفاده می‌شود.

messagea db 0DH,0AH, 'Enter your decimal number: ',0AH,0DH,'\$', 0: یک رشته صفر پایان messagea برای درخواست از کاربر برای وارد کردن یک عدد دسیمال تعریف می‌کند.

numc db buflen dup(' '), '\$': یک بافر 14 بایتی numc با مقدار اولیه فضاها و یک علامت دلار برای ذخیره فرم باینری تعریف می‌کند.

messagec db 0dh,0ah, 'Binary form: ',0dh,0ah,'\$', 0: یک رشته صفر پایان messagec برای نمایش فرم باینری تعریف می‌کند.

```
choice_message db 0dh,0ah, 'Choose 1 for Binary to Decimal , 2 for
Decimal to Binary: $', 0
binary_prompt db 0dh,0ah, "Enter binary number : $", 0
```

choice_message: پیام انتخاب نوع تبدیل. با وارد کردن عدد 1 در داس کاربر می‌تواند عدد باینری را به دسیمال تبدیل کند و با وارد کردن عدد 2 در داس برعکس این کار قابل انجام است.

binary_prompt: پیام درخواست ورودی باینری.

تعریف متغیرهای نتیجه و flag:

```
sum DW 0
flag DB 0
choice DB 0
```

Sum DW 0: یک متغیر دو بایتی برای ذخیره نتیجه تبدیل.

flag DB 0: یک متغیر بایتی برای نشان دادن وضعیت و کنترل خطاها.

choice DB 0: این متغیر بایتی برای ذخیره انتخاب کاربر برای نوع تبدیل مشخص می‌شود.

شروع بخش کد و تعریف پروسیجر اصلی:

```

    .code
main proc

```

بارگذاری بخش داده و نمایش پیام انتخاب:

```

    mov ax, @data
    mov ds, ax

```

mov ax, @data: آدرس شروع بخش داده که در مرحله قبل تایین کردیم را در رجیستر AX بارگذاری می کند.

mov ds, ax: آدرس داده موجود در رجیستر AX به رجیستر DS منتقل می شود.

```

    lea dx, choice_message
    call print_string

```

Lea dx, choice_message: آدرس پیام انتخاب کاربر را در رجیستر DX بارگذاری می کند.

call print_string: پروسیجر print_string که در ادامه به آن خواهیم پرداخت را فراخوانی می کند تا پیام نمایش داده

شود.

دریافت ورودی از کاربر برای انتخاب تبدیل:

```
mov ah, 01h
int 21h
sub al, '0'
mov choice, al
```

mov ah, 01h: آماده‌سازی وقفه 21h داس برای خواندن یک کاراکتر.

int 21h: وقفه داس را برای خواندن کاراکتر از صفحه کلید فراخوانی می‌کند.

sub al, '0': با تفریق ورودی انتخاب کاربر را که به صورت کاراکتر در **al** ذخیره به کاراکتر '0' به مقدار عددی آن تبدیل می‌کنیم.

mov choice, al: مقدار عددی تبدیل شده را در متغیر **choice** ذخیره می‌کند.

بررسی انتخاب کاربر و اجرای پروسیجر مناسب:

```
cmp choice, 1
je inputBinary
cmp choice, 2
je inputDecimal
jmp stop_program
```

cmp choice, 1: مقدار **choice** را با 1 مقایسه می‌کند.

je inputBinary: در صورتی که مقدار ذخیره شده در **choice** برابر با 1 باشد به پروسیجر **inputBinary** می‌پرد.

cmp choice, 2: مقدار **choice** را با 2 مقایسه می‌کند.

je inputDecimal: در صورتی که مقدار ذخیره شده در **choice** برابر با 2 باشد به پروسیجر **inputDecimal** می‌پرد.

jmp stop_program: و در صورتی که مقدار ورودی یا **choice** معتبر نباشد به پروسیجر **stop_program** می‌پرد.

در ادامه به معرفی سه پروسیجر نام برده شده در بالا خواهیم پرداخت.

پروسیجر inputbinary:

نمایش پیام ورودی باینری.

دریافت ورودی باینری با استفاده از پروسیجر GET_STRING.

بررسی صحت ورودی باینری و تبدیل آن به دسیمال.

نمایش نتیجه دسیمال.

```
inputBinary:
    lea dx, binary_prompt
    call print_string

    mov dx, buflen
    lea di, numc
    call GET_STRING

    mov sum, 0

    mov cx, 8
    mov si, OFFSET numc

check_s:
    cmp [si], 0
    jne ok0
    mov flag, 1
    jmp convert
ok0:
    cmp [si], 'b'
    jne ok1
    mov flag, 1
    jmp convert
ok1:
    cmp [si], 31h
    jna ok2
    jmp error_not_valid
ok2:
    inc si
    loop check_s

convert:
    mov bl, 1
    mov cx, si
    sub cx, OFFSET numc
    dec si
    jcxz stop_program

next_digit:
    mov al, [si]
    sub al, 30h
    mul bl
    add sum, ax
    shl bl, 1
    dec si
    loop next_digit
```

inputBinary: لیبل شروع پروسیجر inputBinary.

lea dx, binary_prompt: آدرس پیام ورودی باینری را در رجیستر dx باگذاری می‌کند.

call print_string: فراخوانی پروسیجر **print_string** برای نمایش پیام ورودی باینر که در رجیستر dx باگذاری شده.

mov dx, buflen: تنظیم اندازه بافر یا ورودی با مقدار قبلی **buflen** که همان 10 است.

lea di, numc: آدرس بافر **numc** را در **di** بارگذاری می‌کنیم.

call GET_STRING: پروسیجر **GET_STRING** که در ادامه به معرفی و بررسی آن می‌پردازیم را فراخوانی می‌کنیم.

mov sum, 0: ریست مقدار **sum** برای مطمئن شدن از اینکه مقدار آن صفر است که قرار است در ادامه برای ذخیره مقدار عدد دسیمال حاصل از تبدیل باینری استفاده شود.

در ابتدا بررسی می‌کنیم که ورودی کاربر فقط شامل اعداد باینری (0 و 1) باشد. به این منظور حلقه‌ای با عنوان **check_s** تعریف کردیم:

mov cx, 8: تنظیم شمارنده حلقه که تعریف خواهد شد.

mov si, OFFSET numc: بارگذاری آدرس شروع بافر **numc** در **si**.

check_s: شروع حلقه بررسی صحت ورودی باینری.

cmp [si], 0: مقایسه مقدار با صفر. صفر نشان‌دهنده انتهای رشته (null terminator) است. اگر صفر باشد یعنی به انتهای رشته رسیده‌ایم.

jne ok0: در صورتی که کاراکتر صفر نباشد یعنی به انتهای رشته نرسیده باشیم به لیبل **ok0** پرش خواهیم داشت.

mov flag, 1: در صورتی که کاراکتر صفر باشد یعنی به انتهای رشته رسیده باشیم پرچم **flag** را برابر 1 قرار می‌دهیم و در خط بعدی به لیبل **convert** پرش خواهیم داشت.

jmp convert: پرش به لیبل **convert**.

ok0: در صورتی که مقدار برابر صفر نباشید، این قسمت اجرا می‌شود.

cmp [si], 'b': چون گاهی به عنوان پیشنهاد اعداد باینری از 'b' استفاده می‌شود. کاراکتر فعلی را با این کاراکتر هم مقایسه می‌کنیم.

jne ok1: در صورتی که کاراکتر برابر 'b' باشد، به لیبل **ok1** پرش خواهیم داشت.

mov flag, 1: در صورتی که کاراکتر برابر 'b' باشد، پرچم flag به 1 قرار می‌دهیم و در خط بعدی به لیبل convert پرش خواهیم داشت.

jmp convert: پرش به لیبل convert.

ok1: در صورتی که مقدار برابر 'b' نباشید، این قسمت اجرا می‌شود.

cmp [si], 31h: در این بخش مقدار با 31h مقایسه می‌شود. به دلیل اینکه 31h همان کد کاراکتر "1" در ASCII است مقایسه می‌شود که کاراکتر فعلی صفر یا یک باشد.

jna ok2: در صورتی که کاراکتر از "1" بزرگتر نباشید یعنی همان صفر یا یک باشید، به لیبل ok2 پرش خواهیم داشت.

jmp error_not_valid: در صورتی که کاراکتر بزرگتر از "1" باشد یعنی ورودی نامعتبر است و به لیبل error_not_valid پرش خواهیم داشت تا پیام خطا نمایش داده شود.

ok2: در صورتی که کاراکتر معتبر باشد، این قسمت اجرا می‌شود.

inc si: مقدار si را افزایش می‌دهیم تا به کاراکترها بعدی اشاره کند و با تکرار حلقه همین روند بررسی را برای کاراکترها بعدی اعمال کند.

loop check_s: تکرار حلقه برای بررسی باقی کاراکترها با کاهش مقدار cx تا زمانی که صفر بشود

نکته: توجه می‌کنیم چون مقدار cx در ابتدا هشت قرار داده شده است و در حلقه کاهش پیدا می‌کند، ورودی عدد باینری ما نهایتاً هشت بیت می‌تواند باشد.

Convert: شروع لیبل convert برای تبدیل ورودی باینری به دسیمال.

mov bl, 1: در رجیستر bl مقدار 1 را ذخیره می‌کنیم. bl همان وزن بین فعلی یا مقدار اولیه نشانه‌دهنده 2^0 است که LSB است. در طول حلقه، مقدار این رجیستر با دستور شیفت چپ دو برابر می‌شود.

mov cx, si: مقدار si که در پایان حلقه بررسی قبلی، به انتهای ورودی کاربر رسیده است را به رجیستر cx منتقل می‌کنیم. اکنون cx طول واقعی رشته باینری را نگهداری می‌کند.

Sub cx, OFFSET numc: با کسر آدرس شروع بافر numc از مقدار فعلی cx تعداد کاراکترهای ورودی کاربر محاسبه می‌شود.

مثال اضافه برای توضیح کد بالا: فرض کنیم آدرس شروع numc برابر 0x200 باشد. مقدار فعلی cx برابر 0x208 (انتهای رشته باینری) باشد. نتیجه محاسبه: $0x208 - 0x200 = 8$ (طول رشته).

dec si: si را کاهش می‌دهیم تا به آخرین بین اشاره کند چون قرار است در حلقه‌ای که در ادامه بررسی خواهیم کرد یکی یکی به عقب بررسی کند. این کاهش ضروری است چون در طی اعمال حلقه بررسی، تا کاراکتر انتهای رشته پیش رفته بودیم.

jcxz stop_program: در اینجا بررسی می‌کنیم اگر مقدار **cx** صفر است به لیبل **stop_program** پرش کنیم تا برنامه متوقف شود. این در صورتی پیش می‌آید که کاربر هیچ کاراکتری وارد نکرده باشد. در غیر این صورت عملیات تبدیل اجرا می‌شود.

next_digit: این لیبل شروع یک حلقه برای پردازش و هر رقم از رشته باینری است. توجه کنید که این حلقه از **LSB** شروع و به سمت **MSB** حرکت می‌کند.

mov al, [si]: مقدار ذخیره شده در آدرسی که **si** به آن اشاره می‌کند که همان کاراکتر در آن رشته است را به رجیستر **al** منتقل می‌شود.

sub al, 30h: در اینجا با کسر کردن **30h** از کد **ASCII** کاراکتر فعلی موجود در **al** کاراکتر موجود را به عدد تبدیل می‌کنیم و در **al** باز ذخیره می‌کنیم.

mul bl: در اینجا مقدار عددی فعلی تبدیل شده موجود در **al** در وزن فعلی که در **bl** از قبل ذخیره کرده بودیم را ضرب می‌کنیم و در **ax** ذخیره می‌کنیم.

add sum, ax: در این دستور مقدار ضرب شده در مرحله قبل را با **sum** جمع می‌کنیم تا به تدریج در **sum** مقدار دسیمال معادل رشته باینری وارد شده را محاسبه شود.

shl bl, 1: با شیفت چپ دادن مقدار قبلی **bl** آن را دو برابر می‌کنیم تا وزن معادل هر بیت برای ضرب به دست آید. به ترتیب 2^0 ، 2^1 ، 2^2 و...

dec si: مقدار **si** را کاهش می‌دهیم تا به کاراکترهای قبلی اشاره کند تا اینکه به **MSB** برسیم.

loop next_digit: حلقه تا زمانی که **cx** مساوی صفر نشده باشد یعنی بیت‌هایی برای پردازش باقی‌مانده باشد ادامه می‌یابد.

پروسیجر **print_unsigned**:

هدف: نمایش عدد دسیمال محاسبه شده.

خروجی: چاپ عدد دسیمال.

```
print_unsigned:
    call print
    db 0dh, 0ah, "Decimal: $", 0
    mov ax, sum
    call PRINT_NUM_UNS
    jmp stop_program
```

print_unsigned: لیبل شروع پروسیجر **print_unsigned**.

call print: پروسیجر **print** که در ادامه معرفی می کنیم را فراخوانی می کند.

db 0dh, 0ah, "Decimal: \$", 0: نمایش رشته "Decimal: "

mov ax, sum: مقدار عدد محاسبه شده که در **sum** ذخیره کردیم به رجیستر **ax** منتقل می شود. برای نمایش به کار خواهد رفت.

call PRINT_NUM_UNS: پروسیجر **PRINT_NUM_UNS** که در ادامه معرفی می کنیم را فراخوانی می کند.

jmp stop_program: بعد از اجرای برنامه با این دستور به لیبل **stop_program** منتقل می شود تا به کاربر اطلاع دهد که برنامه پایان یافته و منتظر بماند با فشار دادن یک کلید توسط کاربر برنامه به طور کامل متوقف شود.

پروسیجر **inputDecimal**

هدف : دریافت ورودی دسیمال از کاربر و تبدیل آن به باینری.

خروجی : نمایش فرم باینری معادل ورودی دسیمال

```
inputDecimal:
    lea dx, messagea
    call print_string
    lea dx, maxlen
    call get_decimal_input
    lea si, numa
    call asctobin
    lea si, numc
    call bintoasc
    lea dx, messagec
    call print_string
    lea dx, numc
    call print_string
    jmp stop_program
```

inputDecimal: لیبل شروع پروسیجر **inputDecimal**.

lea dx, messagea: آدرس پیامی که در **messagea** در ابتدا تایین کردیم را در رجیستر **dx** بارگزاری می کنیم.

call print_string: با فراخوانی پروسیجر **print_string**. رشته پیامی که به **dx** منتقل شده را در داس نمایش می دهد و از کاربر خواسته می شود تا عدد دسیمال خود را وارد کند.

lea dx, maxlen: آدرس متغیر **maxlen** را در رجیستر **dx** بارگزاری می کنیم تا مشخص شود طول عدد وارد شده چقدر می تواند باشد. که در اینجا حداکثر 5 رقم می تواند باشد طبق چیزی که تایین شده.

call get_decimal_input: با فراخوانی پروسیجر `get_decimal_input` که در ادامه به بررسی آن خواهیم پرداخت عدد وارد

شده توسط کاربر دریافت شده و به صورت رشته در `numa` ذخیره می‌شود.

lea si, numa: آدرس `numa` که آدرس همان رشته عدد وارد شده توسط کاربر است به `si` منتقل می‌شود.

call ascbin: فراخوانی پروسیجر `ascbin` برای تبدیل عدد دسیمال به باینری.

lea si, numc: آدرس `numc` که همان محل ذخیره عدد باینری معادل است را در `si` قرار می‌دهد. نکته اینکه `numc` همان محلی است که عدد باینری معادل ذخیره خواهد شد.

call bintoasc: فراخوانی پروسیجر `bintoasc` برای تبدیل عدد باینری تبدیل شده به رشته.

lea dx, message: آدرس متغیر `message` که حاوی پیامی است که به کاربر اطلاع می‌دهد معادل باینری در حال نمایش است، را در رجیستر `dx` قرار بارگزاری می‌کند.

call print_string: فراخوانی پروسیجر `print_string` برای چاپ محتوی پیام موجود در `message`.

lea dx, numc: آدرس موجود در `numc` که همان محل ذخیره عدد باینری است در رجیستر `dx` بارگزاری می‌شود.

call print_string: با فراخوانی این رویه، عدد باینری ذخیره شده در `numc` روی صفحه نمایش چاپ خواهد شد.

jmp stop_program: بعد از اجرای این دستورات در آخر به پروسیجر `stop_program` پرش خواهیم داشت.

نکته: در ادامه به معرفی و بررسی پروسیجرهای `stop_program` ، `print_string` ، `bintoasc` ، `ascbin` و `get_decimal_input` خواهیم پرداخت.

پروسیجر `error_not_valid`:

هدف : نمایش پیام خطا در صورت ورودی باینری نامعتبر.

خروجی : نمایش پیام خطا "Error: Only zeros and ones are allowed!" .

```
error_not_valid:
    call print
    db 0dh, 0ah, "Error: Only zeros and ones are allowed!$", 0
```

error_not_valid: لیبل شروع پروسیجر `error_not_valid` که وظیفه دارد در صورت نامعتبر بودن ورودی پیام مشخصی را

چاپ کند. از این لیبل در کدهای قبلی بعد از دستور جامپ استفاده می‌شد.

call print: برای چاپ کردن پیام مورد نظر پروسیجر `print` را فراخوانی می‌کنیم.

`db 0dh, 0ah, "Error: Only zeros and ones are allowed!$", 0` رشته همان پیامی از داده‌ها که توسط پروسیجر `print` چاپ خواهد شد.

پروسیجر `stop_program`:

هدف: نمایش پیام پایان برنامه و انتظار فشار کلید.

خروجی: نمایش پیام "Press any key..." و پایان برنامه.



```
stop_program:
    call print
    db 0dh, 0ah, "Press any key...$", 0
    mov ah, 0
    int 16h
    mov ah, 4ch
    int 21h
    ret
```

Stop_program: لیبل شروع پروسیجر `Stop_program` که وظیفه دارد اجرای برنامه را به پایان برساند. از این لیبل در کدهای قبلی بعد از دستور جامپ استفاده می‌شد.

call print: برای چاپ کردن پیام مورد نظر پروسیجر `print` را فراخوانی می‌کنیم.

`db 0dh, 0ah, "Press any key...$", 0` رشته همان پیامی از داده‌ها که توسط پروسیجر `print` چاپ خواهد شد.

`mov ah, 0`: تنظیم AH برای انتظار فشردن یک کلید توسط کاربر.

`int 16h`: وقفه برای انتظار فشردن یک کلید توسط کاربر.

`mov ah, 4ch`: تنظیم AH برای پایان برنامه.

`int 21h`: وقفه برای پایان برنامه.

`Ret`: برگشت از پروسیجر.

پروسیجر `print_string`:

رشته‌ای که در آدرس `dx` قرار دارد، چاپ می‌شود.

رشته باید با `$` تمام شود.

پس از چاپ، اجرای برنامه به ادامه دستوراتی که رویه را فراخوانی کرده‌اند برمی‌گردد.

```
print_string proc
    mov ah, 9h
    int 21h
    ret
print_string endp
```

`print_string proc`: شروع پروسیجر `print_string`.

`mov ah, 9h`: تنظیم `AH` برای نمایش رشته مورد نظر.

`int 21h`: وقفه برای نمایش رشته مورد نظر.

`Ret`: برگشت به پروسیجری که در آن پروسیجر `print_string` فراخوانی شده.

`Print_string endp`: دستور اتمام پروسیجر.

پروسیجر `get_decimal_input`:

هدف: دریافت ورودی دسیمال از کاربر.

خروجی: ذخیره ورودی دسیمال در `numa`.

```
get_decimal_input proc
    mov ah, 0Ah
    lea dx, maxlen
    int 21h
    ret
get_decimal_input endp
```

get_decimal_input proc: شروع پروسیجر **get_decimal_input**.

mov ah, 0Ah: تنظیم AH برای دریافت ورودی رشته از کاربر.

lea dx, maxlen: بارگذاری آدرس **maxlen** در رجیستر **dx**.

int 21h: وقفه برای دریافت ورودی رشته از کاربر.

Ret: برگشت به پروسیجری که در آن پروسیجر **get_decimal_input** فراخوانی شده.

get_decimal_input endp: دستور اتمام پروسیجر.

پروسیجر **asctobin**:

هدف: تبدیل عدد دسیمال (وارد شده به صورت کاراکترهای ASCII) به باینری.

خروجی: ذخیره عدد باینری در **ax**.

```
asctobin proc
    mov ax, 0
count:
    mov bl, [si]
    cmp bl, 0dh
    je end_convert
    mov dx, 10
    mul dx
    sub bl, '0'
    mov bh, 0
    add ax, bx
    inc si
    jmp count
end_convert:
    ret
asctobin endp
```

asctobin proc: شروع پروسیجر **asctobin**.

mov ax, 0: رجیستر **AX** را با 0 مقداردهی می‌کنیم به منظور ذخیره مقدار نهایی دسیمال.

در ادامه برای بررسی هر کاراکتر در رشته ورودی حلقه‌ای آورده می‌شود.

Count: شروع حلقه بررسی.

mov bl, [si]: مقادیر موجود در آدرس si که به اولین کاراکتر رشته ورودی اشاره می‌کند به رجیستر bl برای تبدیل منتقل می‌شود.

cmp bl, 0dh: این دستور برای بررسی پایان رشته است. در سیستم ASCII، کاراکتر 0dh معادل CR است که معمولاً در انتهای رشته ورودی توسط کاربر قرار می‌گیرد.

je end_convert: در صورتی که مقدار bl با 0dh برابر باشد یعنی به انتهای رشته رسیدیم و حلقه باید متوقف شود لذا در این صورت به لیبل end_convert پرش خواهیم داشت تا پروسیجر خاتمه یابد.

mov dx, 10: مقدار 10 را در رجیستر dx قرار می‌دهیم. از این رجیستر برای ضرب در هر کاراکتر دسیمال به منظور قرارگیری درست مکان هر کاراکتر در عدد دسیمال جدید.

mul dx: در اینجا با ضرب ax در dx که همان مقدار 10 را دارد که هدفی که توضیح داده شد می‌رسیم. به این صورت که با این ضرب تمام ارقام قبلی یکی به سمت چپ منتقل شوند تا فضا برای ارقام جدید ایجاد شود.

sub bl, '0': به منظور تبدیل کاراکتر به عدد مورد نظر.

mov bh, 0: تنظیم BH به 0 جهت اطمینان از اینکه هیچ داده اضافه‌ای در آن ذخیره نشده باشد.

add ax, bx: عدد معادل دسیمال کاراکتر فعلی که در رجیستر BX قرار دارد را با نتیجه ذخیره شده قبلی در رجیستر AX جمع می‌کند و در ax قرار می‌دهد.

inc si: مقدار SI را یکی افزایش می‌دهد تا به اشاره‌گر به کاراکتر بعدی رشته برود.

jmp count: حلقه تا زمانی که به پایان رشته وارد شده برسیم ادامه پیدا می‌کند.

end_convert: لیبل به منظور پایان تبدیل.

Ret: برگشت به پروسیجری که در آن پروسیجر asctobin فراخوانی شده.

asctobin endp: دستور اتمام پروسیجر.

هدف : تبدیل باینری به رشته.

خروجی : ذخیره رشته باینری در numc.

```
bintoasc proc
    mov bx, 2
    add si, buflen-1
cont_i:
    mov dx, 0
    div bx
    add dl, '0'
    mov [si], dl
    cmp ax, 0
    je end_convert2
    dec si
    jmp cont_i
end_convert2:
    ret
bintoasc endp
```

bintoasc proc: شروع پروسیجر bintoasc.

mov bx, 2: در رجیستر bx مقدار 2 را قرار می‌دهیم. به این دلیل که برای تبدیل باید از 2 به عنوان مقسوم‌علیه در عملیات تقسیم استفاده خواهیم کرد.

add si, buflen-1: در این دستور مقدار si به آخرین موقعیت در رشته یا بافر تنظیم می‌شود که قرار است عدد باینری معادل در آن نوشته شود. buflen طول بافر ورودی است که به عنوان مقدار ثابت در ابتدای کد تعریف شده است. با کم کردن 1 از si، buflen به آخرین خانه بافر منتقل می‌شود. این عمل به این دلیل انجام می‌شود که در ادامه، عدد باینری به صورت معکوس (از کمترین بیت به بیشترین بیت) در بافر ذخیره می‌شود. بنابراین باید از انتهای بافر شروع به نوشتن کنیم.

در ادامه حلقه‌ای تعریف می‌کنیم که در هر تکرار، یک بیت از عدد ذخیره شده در ax را استخراج کرده و به معادل کاراکتری آن در رشته تبدیل می‌کند.

Conti: شروع حلقه استخراج و تبدیل عدد به رشته.

mov dx, 0: مقدار 0 را در رجیستر dx قرار می‌دهیم. به این دلیل که با هر با تقسیم در خط بعدی، باقی‌مانده تقسیم در dx قرار می‌گیرد. لذا باید در ابتدا برابر مقدار صفر قرار دهیم تا فقط باقی‌مانده‌های تقسیم در آن ذخیره شود.

div bx: عملیات تقسیم عدد ذخیره شده در رجیستر ax به رجیستر bx که در آن از قبل مقدار 2 را قرار داده بودیم. با هر بار تقسیم باقی‌مانده که یا 0 است یا 1 در رجیستر dx و حاصل تقسیم در ax ذخیره می‌شود تا برای عملیات تقسیم بعدی استفاده شود.

add dl, '0': چون بعد از هر بار تقسیم باقی‌مانده در dl قرار می‌گیرد چون می‌خواهیم عدد را به کاراکتر ASCII تبدیل کنیم، عدد را با "0" جمع می‌کنیم. در نتیجه اگر باقی‌مانده 0 باشد به "0" و اگر 1 باشد به "1" تبدیل می‌شود و در dl ذخیره می‌شود.

mov [si], dl: در این مرحله مقدار درون dl را که تبدیل شده باقی‌مانده است در موقعیت فعلی اشاره گر si ذخیره می‌شود. می‌دانیم که در ابتدا si به آخرین خانه بافر اشاره می‌کند و از آنجا به عقب حرکت می‌کند.

cmp ax, 0: مقدار درون رجیستر ax را با 0 مقایسه می‌کنیم. در صورتی که مقدار موجود برابر صفر باشد، به این معنی است که تمام بیت‌های عدد باینری پردازش شده‌اند.

je end_convert2: در صورتی که ax برابر با صفر باشد و بیت دیگری برای پردازش نمانده باشد، حلقه متوقف شده و بر لیبل end_convert2 که در ادامه توضیح خواهیم داد پرش خواهیم داشت.

dec si: در صورتی که هنوز بیت دیگری برای پردازش نمانده باشد، مقدار si کاهش میابد و به سمت شروع بافر حرکت می‌کند تا مکان بعدی برای ذخیره کاراکتر بعدی فراهم شود.

jmp conti: تکرار حلقه بررسی و تبدیل.

end_convert2: پایان حلقه بررسی و تبدیل در صورت پردازش شدن همه‌ی بیت‌ها.

Ret: برگشت به پروسیجری که در آن پروسیجر bintoasc فراخوانی شده.

Bintoasc endp: دستور اتمام پروسیجر.

پروسیجر GET_STRING:

هدف : دریافت ورودی رشته‌ای از کاربر.

خروجی : ذخیره ورودی رشته‌ای در بافر numc.

```
GET_STRING PROC NEAR
    PUSH AX
    PUSH CX
    PUSH DI
    PUSH DX
    MOV CX, 0
    CMP DX, 1
    JBE empty_buffer
    DEC DX
wait_for_key:
    MOV AH, 0
    INT 16h
    CMP AL, 13
    JZ end_convert3
    CMP AL, 8
    JNE add_to_buffer
    JCXZ wait_for_key
    DEC CX
    DEC DI
    PUTC 8
    PUTC ' '
    PUTC 8
    JMP wait_for_key
add_to_buffer:
    CMP CX, 8
    JAE wait_for_key
    MOV [DI], AL
    INC DI
    INC CX
    MOV AH, 0Eh
    INT 10h
    JMP wait_for_key
end_convert3:
    MOV [DI], 0
empty_buffer:
    POP DX
    POP DI
    POP CX
    POP AX
    RET
GET_STRING ENDP
```

GET_STRING PROC NEAR: شروع پروسیجر محلی GET_STRING.

PUSH AX, PUSH CX, PUSH DI, PUSH DX: ذخیره مقادیر رجیسترها در پشته. به منظور حفظ مقادیر اصلی رجیسترهایی که در داخل این پروسیجر تغییر خواهند کرد از دستور push استفاده شده است. این تضمین می‌کند که بعد از پایان پروسیجر، مقادیر اصلی بازگردانده شوند

MOV CX, 0: شمارنده کاراکترهای وارد شده را با صفر مقداردهی می‌کند.

CMP DX, 1: در اینجا بررسی می‌شود که آیا طول بافری که در رجیستر dx قرار دارد کمتر از 2 است یا خیر.

JBE empty_buffer: در صورتی که طول بافردرون رجیستر DX کمتر یا برابر با 1 باشد به منظور خالی کردن بافر و خروج به لیبل empty_buffer پرش خواهیم داشت.

DEC DX: به منظور رزرو کردن یک فضای اضافی برای رشته 0، از طول بافر یکی کم می‌شود.

در ادامه حلقه‌ای برای انتظار برای ورود کلید توسط کاربر بررسی خواهد شد.

wait_for_key: لیبل برای شروع حلقه انتظار برای ورود کلید توسط کاربر.

MOV AH, 0 و INT 16h: منتظر می‌ماند تا کاربر یک کلید را فشار دهد و مقدار کلید فشرده شده را در رجیستر al ذخیره کند.

CMP AL, 13: بررسی می‌شود که آیا کاراکتر وارد شده توسط کاربر برابر با 13 که همان کد کلید Enter می‌شود است یا خیر.

JZ end_convert3: در صورتی که کاراکتر وارد شده کلید Enter باشد به معنی پایان دریافت رشته است و به این منظور به لیبل end_convert3 پرش خواهیم داشت.

CMP AL, 8: بررسی می‌شود که آیا کاراکتر وارد شده توسط کاربر برابر با 8 که همان کد کلید Backspace می‌شود است یا خیر.

JNE add_to_buffer: در صورتی که کاراکتر وارد شده کلید Backspace نباشد، به منظور اضافه کردن کاراکتر وارد شده با بافر به لیبل add_to_buffer پرش خواهیم داشت.

JCXZ wait_for_key: اگر مقدار درون رجیستر cx برابر با 0 بود که یعنی هیچ کاراکتری در بافر نیست، منتظر ورودی جدید خواهیم ماند.

در صورتی که کاراکتر وارد شده کلید Backspace باشد:

DEC CX: شمارنده کاراکترهای وارد شده CX کاهش می‌یابد.

DEC DI: اشاره‌گر بافر di به کاراکتر قبلی برمی‌گردد.

کاراکتر Backspace را روی صفحه نمایش می‌دهد تا به کاربر نشان دهد یک کاراکتر حذف شده است :

PUTC 8: چاپ کاراکتر Backspace.

PUTC ' ': چاپ فضای خالی.

PUTC 8: چاپ کاراکتر Backspace.

JMP wait_for_key: تکرار حلقه تا دریافت کاراکتر جدید.

add_to_buffer: لیبل شروع پروسیجر add_to_buffer که به منظور افزودن کاراکتر به بافر از آن استفاده خواهیم کرد.

CMP CX, 8: بررسی می‌شود که آیا تعداد کاراکترهای وارد شده به حداکثر طول مجاز که در اینجا 8 کاراکتر است رسیده یا خیر/

JAE wait_for_key: در صورتی که طول رشته به حداکثر رسیده باشد، به حلقه قبلی برمی‌گردد ولی ورودی‌های اضافی نادیده گرفته می‌شوند.

MOV [DI], AL: کاراکتر وارد شده را در بافر که همان محل اشاره‌شده توسط DI است ذخیره می‌کند.

INC DI: افزایش مقدار شماره‌گر بافر DI.

INC CX: افزایش مقدار شمارنده کاراکترهای وارد شده CX.

MOV AH, 0Eh: تنظیم مقدار AH برای نمایش کاراکتر وارد شده.

INT 10h: وقفه برای نمایش کاراکتر وارد شده.

JMP wait_for_key: پرش به حلقه wait_for_key.

end_convert3: لیبل شروع پروسیجر end_convert3 که به منظور پایان تبدیل مورد استفاده قرار خواهد گرفت.

MOV [DI], 0: مقدار صفر که همان ترمیناتور رشته است را در محل فعلی اشاره‌گر DI قرار می‌دهد تا پایان رشته مشخص شود.

empty_buffer: لیبل شروع پروسیجر empty_buffer برای زمانی که بافر خالی است.

POP DX, POP DI, POP CX, POP AX: بازیابی مقادیر رجیسترها از پشته.

RET: برگشت به پروسیجری که در آن پروسیجر GET_STRING فراخوانی شده.

GET_STRING ENDP: دستور اتمام پروسیجر.

PRINT_NUM پروسیجر

هدف : نمایش عدد، با بررسی مثبت یا منفی بودن آن.

خروجی : چاپ عدد.

```
PRINT_NUM PROC NEAR
    PUSH DX
    PUSH AX
    CMP AX, 0
    JNZ not_zero
    PUTC '0'
    JMP printed_pn
not_zero:
    CMP AX, 0
    JNS positive
    NEG AX
    PUTC '-'
positive:
    CALL PRINT_NUM_UNSP
printed_pn:
    POP AX
    POP DX
    RET
PRINT_NUM ENDP
```

PRINT_NUM PROC NEAR: شروع پروسیجر محلی PRINT_NUM.

PUSH DX, PUSH AX: مقادیر رجیسترهای AX و DX روی پشته ذخیره می‌شوند تا هنگام اتمام پروسیجر، مقادیر اصلی بازگردانده شوند. (همانند پروسیجر GET_STRING که پیش‌تر بررسی شد).

نکته: AX شامل عدد ورودی خواهد بود و BX برای اطمینان از عدم تغییر در سایر بخش‌های برنامه ذخیره می‌شود.

CMP AX, 0: مقدار ذخیره شده در رجیستر AX با 0 مقایسه می‌شود.

JNZ not_zero: در صورتی که برابر صفر نباشید به لیبل not_zero پرش خواهیم داشت.

PUTC '0': در صورتی که برابر صفر باشد، کاراکتر "0" در خروجی چاپ خواهد شد.

JMP printed_pn: پس از چاپ صفر در خروجی به لیبل printed_pn پرش خواهیم داشت تا مقادیر ذخیره شده رو پشته بازگردانده شود و از رویه خارج شود.

not_zero: لیبل شروع پروسیجر not_zero برای حالت غیر صفر.

CMP AX, 0: در اینجا مقدار ذخیره شده در رجیستر AX بار دیگری بررسی می‌شود تا منفی یا مثبت بودن آن مشخص شود.

JNS positive: در صورتی که مقدار ذخیره شده مثبت یا صفر باشد به لیبل positive پرش خواهیم داشت.

NEG AX: در صورتی که مقدار ذخیره شده منفی باشد، با دستور NEG مقدار تبدیل به یک عدد مثبت می‌شود.

PUTC '-': چاپ علامت منفی در خروجی.

Positive: لیبل شروع پروسیجر positive برای حالت مثبت.

CALL PRINT_NUM_UN: پس از چاپ علامت البته در صورت وجود، پروسیجر PRINT_NUM_UN به منظور چاپ عدد مثبت موردنظر فراخوانی می‌شود.

printed_pn: لیبل شروع پروسیجر printed_pn برای پایان عملیات چاپ.

POP AX, POP DX: بازبازی مقادیر رجیسترها از پشته.

RET: برگشت به پروسیجر PRINT_NUM فراخوانی شده.

PRINT_NUM ENDP: دستور اتمام پروسیجر.

پروسیجر PRINT_NUM_UN\$:

هدف : نمایش عدد بدون علامت.

خروجی : چاپ ارقام عدد به ترتیب.

```
PRINT_NUM_UN$ PROC NEAR
    PUSH AX
    PUSH BX
    PUSH CX
    PUSH DX
    MOV CX, 1
    MOV BX, 10000
    CMP AX, 0
    JZ print_zero
begin_print:
    CMP BX, 0
    JZ end_print
    CMP CX, 0
    JE calc
    CMP AX, BX
    JB skip
calc:
    MOV CX, 0
    MOV DX, 0
    DIV BX
    ADD AL, 30h
    PUTC AL
    MOV AX, DX
skip:
    PUSH AX
    MOV DX, 0
    MOV AX, BX
    DIV CS:ten
    MOV BX, AX
    POP AX
    JMP begin_print
print_zero:
    PUTC '0'
end_print:
    POP DX
    POP CX
    POP BX
    POP AX
    RET
ten DW 10
PRINT_NUM_UN$ ENDP
```

PRINT_NUM_UN\$ PROC NEAR: شروع پروسیجر محلی PRINT_NUM_UN\$.

PUSH AX, BX, CX, DX: مقادیر رجیسترهای AX, BX, CX, و DX روی پشته ذخیره می‌شوند تا تغییرات در این رجیسترها برنامه اصلی را تحت تأثیر قرار ندهند. (مانند پروسیجرهای محلی‌ای که در گذشته آورده شده بود)

MOV CX, 1: مقدار 1 را در رجیستر CX قرار می‌دهیم. CX یک نشانگر است که مشخص می‌کند آیا تاکنون رقمی چاپ شده است یا خیر. در ابتدا فرض می‌شود که هیچ رقمی چاپ نشده است.

MOV BX, 10000: مقدار 10000 را در رجیستر BX قرار می‌دهیم. این مقدار همان مقدار تقسیم‌کننده است که عدد ذخیره شده در AX به این مقدار تقسیم خواهد شد تا رقم بزرگ‌ترین مرتبه عدد محاسبه شود. این مقدار معادل 10000 است. (بالاترین مرتبه در عدد 5 رقمی)

CMP AX, 0: مقدار رجیستر AX را با 0 مقایسه می‌شود.

JZ print_zero: در صورتی که مقدار AX برابر با صفر باشد، به لیبل **print_zero** پرش خواهیم داشت و کاراکتر "0" چاپ خواهد شد.

در ادامه حلقه‌ای برای چاپ اعداد بررسی خواهد شد.

begin_print: لیبل شروع حلقه چاپ اعداد.

CMP BX, 0: مقدار BX را با 0 مقایسه می‌کند. به عبارتی بررسی می‌کند آیا مقدار تقسیم‌کننده صفر شده است یا خیر.

JZ end_print: اگر BX برابر صفر باشد، تمام ارقام چاپ شده‌اند و به لیبل **end_print** پرش خواهیم داشت.

CMP CX, 0: مقدار رجیستر CX را با 0 مقایسه می‌شود.

JE calc: در صورتی که مقدار رجیستر CX برابر با صفر باشد یعنی یک رقم قبلاً چاپ شده و به لیبل **calc** برای محاسبات پرش خواهیم داشت.

CMP AX, BX: مقدار رجیستر AX را با مقدار رجیستر BX مقایسه می‌شود.

JB skip: در صورتی که AX کوچکتر از BX باشد چون رقم این مرتبه برابر سفر می‌شود، به لیبل **skip** پرش خواهیم داشت.

calc: لیبل شروع پروسسور **calc** به منظور محاسبه.

MOV CX, 0: مشخص می‌شود که یک رقم چاپ شده است.

MOV DX, 0: قبل از مرحله تقسیم مقدار 0 را در رجیستر DX قرار می‌دهیم.

DIV BX: مقدار رجیستر AX را بر مقدار رجیستر BX تقسیم می‌شود. توجه کنید که خارج قسمت در رجیستر AL و باقی‌مانده در رجیستر DX ذخیره می‌شود. خارج قسمت همان رقم فعلی است.

ADD AL, 30h: با جمع رقم فعلی با 30h، رقم عدد را به کد ASCII یا کاراکتر تبدیل می‌کنیم.

PUTC AL: کاراکتر رقم فعلی چاپ می‌شود.

MOV AX, DX: مقدار باقی‌مانده که همان رقم‌های بعدی است به رجیستر AX منتقل می‌شود تا برای مرتبه بعدی پردازش شود.

Skip: لیبل ادامه حلقه چاپ اعداد.

PUSH AX: مقدار رجیستر AX روی پشته ذخیره می‌شود.

MOV DX, 0: مقدار 0 را در رجیستر DX قرار می‌دهیم.

MOV AX, BX: مقدار رجیستر BX به رجیستر AX منتقل می‌شود.

DIV CS:ten: مقدار درون رجیستر AX که همان تقسیم‌کننده است بر 10 تقسیم می‌شود تا مرتبه آن کاهش یابد.

MOV BX, AX: مقدار رجیستر AX که همان مقدار جدید تقسیم‌کننده است به BX منتقل می‌شود.

POP AX: مقدار اصلی رجیستر AX را از پشته بازگردانده می‌شود.

JMP begin_print: پرش به لیبل begin_print برای ادامه حلقه چاپ اعداد.

print_zero: لیبل شروع پروسیجر print_zero به منظور چاپ کاراکتر صفر.

PUTC '0': کاراکتر "0" را چاپ می‌کند.

end_print: لیبل شروع پروسیجر end_print به منظور پایان چاپ.

POP DX, CX, BX, AX: مقادیر رجیسترهایی که در ابتدای رویه ذخیره شده بودند، بازیابی می‌شوند.

RET: برگشت به پروسیجری که در آن پروسیجر PRINT_NUM_UN\$ فراخوانی شده.

ten DW 10: تعریف مقدار 10 در بخش داده‌ها.

PRINT_NUM_UN\$ ENDP: دستور اتمام پروسیجر.

پروسیجر print :

هدف : نمایش کاراکترهای یک رشته.

خروجی : چاپ کاراکترهای رشته تا رسیدن به کاراکتر پایان.

```
print PROC
    MOV CS:temp1, SI
    POP SI
    PUSH AX
next_char:
    MOV AL, CS:[SI]
    INC SI
    CMP AL, 0
    JZ printed_ok
    MOV AH, 0Eh
    INT 10h
    JMP next_char
printed_ok:
    POP AX
    PUSH SI
    MOV SI, CS:temp1
    RET
temp1 DW ?
print ENDP

end main
```

print PROC: لیبل شروع پروسیجر **print**.

MOV CS:temp1, SI: مقدار فعلی رجیستر SI که آدرس رشته‌ای است که قرار است چاپ شود در خود نگه می‌دارد را در متغیر temp1 ذخیره می‌شود. این کار برای بازگرداندن مقدار اصلی SI پس از اجرای پروسیجر انجام می‌شود.

POP SI: مقدار جدیدی از پشته به رجیستر SI منتقل می‌شود. این مقدار، آدرس رشته‌ای است که باید پردازش و چاپ شود.

PUSH AX: مقدار رجیستر AX ذخیره می‌شود، زیرا در این پروسیجر برای چاپ کاراکترها تغییر خواهد کرد.

در ادامه حلقه‌ای را بررسی می‌کنیم که برای پردازش و چاپ کاراکترها نوشته شده است:

next_char: لیبل شروع حلقه پردازش و چاپ کاراکترها.

MOV AL, CS:[SI]: کاراکتر فعلی که در آدرسی که SI نشان می‌دهد ذخیره شده، به رجیستر AL منتقل می‌شود.

INC SI: مقدار رجیستر SI را افزایش می‌یابد تا به کاراکتر بعدی در رشته اشاره کند.

CMP AL, 0: کاراکتر فعلی منتقل شده به رجیستر AL با مقدار صفر که همان null terminator است مقایسه می‌شود.

JZ printed_ok: در صورتی که مقدار رجیستر AL برابر صفر باشد، نشان‌دهنده پایان رشته است و به لیبل **printed_ok** پرش خواهیم داشت.

MOV AH, 0Eh: تابع نمایش کاراکتر را تنظیم می‌کند.

INT 10h: وقفه برای چاپ کاراکتر.

JMP next_char: حلقه تکرار می‌شود و به پردازش کاراکتر بعدی ادامه می‌دهد.

printed_ok: لیبل پروسیجر **printed_ok** برای پایان چاپ کاراکترها و بازگرداندن مقادیر اصلی.

POP AX: مقدار اصلی AX که قبلاً ذخیره شده بود، بازیابی می‌شود.

PUSH SI: مقدار اصلی SI روی پشته ذخیره می‌شود. (برای همگام‌سازی آدرس رشته با وضعیت قبلی)

MOV SI, CS:temp1: مقدار اصلی SI که در ابتدای رویه ذخیره شده بود، به SI بازگردانده می‌شود.

RET: برگشت به پروسیجری که در آن پروسیجر **print** فراخوانی شده.

temp1 DW ?: متغیر temp1 یک فضای 16 بیتی در بخش کد (CS) برای ذخیره مقدار اصلی رجیستر SI است. این متغیر به

ما اجازه می‌دهد مقدار SI را برای بازیابی پس از اتمام عملیات ذخیره کنیم.

print ENDP: دستور اتمام پروسیجر.

end main: دستور برای اتمام بخش کد.

«بخش چهارم»

منابع و مراجع

کتاب زبان اسمبلی: برنامه‌نویسی کامپیوترهای آی. بی. ام. x8086 و سازگار با آن به نوشتار

محمد علی مزیدی، سپیدنام – مشهدی باغانی، ۱۳۷۹

کتاب برنامه‌نویسی و زبان اسمبلی کامپیوترهای شخصی به نوشتار پیتر ایبل، ترجمه فرزانه

کیمیایی – مشهد: انتشارات نما، دروش، ۱۳۷۸

[وبسایت ویکی‌پدیا](#)

[وبسایت فرادرس](#)

[وبسایت کوئرا](#)

[وبسایت ربات‌سازان](#)

[OpenAI](#)

[GitHub](#)

[YouTube](#)

[دوره آموزش برنامه‌نویسی اسمبلی - جامع و کاربردی وبسایت فرادرس](#)